WEEK 13

# Timers, Part 2: PWM, Input Capture, and the Watchdog

The channels. Capturing the counter when the world does something, comparing it to drive an output, and the timer whose job is to notice your program died

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 7, printed pages 213–219 and 225–234.
Lab manual: Lab 05 (PWM motor control), Lab 06 (capture/compare, frequency measurement).
Note: your syllabus lists the watchdog as 7.7; the book prints it earlier, at 213–219.

## 1 What we are actually after this week

Last week we built a counter that counts. It could overflow and interrupt, and that was useful, but it could not interact with the outside world at all.

This week the **channels**, which are what make a timer a peripheral rather than an alarm clock. Each channel does one of two things:

- **Input capture**: the world does something, and the hardware records *when*.
- **Output compare**: the counter reaches a value, and the hardware changes *a pin*.

Both directions, no CPU involvement at the critical moment. That last clause is the entire value proposition.

The plan:

1. Channel mechanics: `CCRy`, the mode select, and the status flags.
2. **Input capture**, and why it is more accurate than an interrupt that reads the counter.
3. The anemometer, worked properly, including the **16-bit overflow problem** and its fix.
4. **Output compare** mode, briefly.
5. **PWM**: edge-aligned and centre-aligned, the duty-cycle arithmetic, and a full worked configuration.
6. The **watchdog**: what it is for, the windowed variant, the timeout formula, and the best practices that are more interesting than they sound.

Item 5 is Lab 05 and your self-balancing robot. Item 3 contains a bug class worth meeting deliberately rather than discovering. Item 6's best practices produce short-answer exam questions and are genuinely good engineering advice.

## 2 Deep dive 1: Channel mechanics

Each timer has multiple channels, used independently. Each channel $y$ has:

- A **capture/compare register** `TIMx_CCRy`. In capture mode the hardware writes it; in compare mode you write it.
- A share of a **control register**: channels 1 and 2 share `TIMx_CCMR1`, channels 3 and 4 share `TIMx_CCMR2`.

- Two flags in `TIMx_SR`: `CCyIF` for the event, `CCyOF` for overcapture.
- An enable bit in `TIMx_CCER`, plus polarity control.
- An interrupt enable `CCyIE` and a DMA enable `CCyDE` in `TIMx_DIER`.

The `CCyS` field in `CCMR` selects input capture or output compare, and **the remaining fields of that register mean completely different things depending on which you chose**. In output mode the bits are `OCyM`, `OCyPE`, `OCyFE`; in input mode the same bit positions are `ICyF` and `ICyPSC`.

**Beware the trap**

That register reuse is a real source of confusion when reading a reference manual. The bit diagram has **two rows**, one for output mode and one for input mode, and if you read the wrong row you will configure a filter setting while believing you set a PWM mode.

When you look up a `CCMR` field, check first which mode row you are in. This is also why HAL has separate `HAL_TIM_PWM_ConfigChannel` and `HAL_TIM_IC_ConfigChannel` functions rather than one generic one.

All the flags live together in `TIMx_SR`: `UIF` for update, `TIF` for trigger, and `CCyIF/CCyOF` per channel. Grouping them means one ISR can identify its cause with a single read, which recall from Week 08 is exactly what a shared handler needs.

## 3 Deep dive 2: Input capture

**Input capture**

Configure a channel to watch an input for a specific transition: rising edge, falling edge, or either. When it occurs, the hardware **copies** `TIMx_CNT` **into** `TIMx_CCRy` and sets `CCyIF`. An interrupt follows if `CCyIE` is set.

The point: **the capture happens in hardware, with no software intervention**, so the recorded time is exact regardless of how long the CPU takes to get around to reading it.

Here is why that matters, and it is the whole argument for the feature.

Suppose instead you took an EXTI interrupt on the edge and read `TIM3->CNT` inside the handler, as in Week 07. How accurate is that reading? It is the true edge time *plus* the interrupt latency, plus any time spent finishing the current instruction, plus any delay if a higher-priority interrupt was running, plus any time interrupts happened to be masked. Recall Week 07's sixteen cycles and Week 08's warning about critical section length: all of that lands directly in your measurement as error, and it **varies from edge to edge**, so it is jitter rather than a constant offset you could calibrate out.

With input capture, the timestamp is latched by hardware at the instant of the edge. The ISR can arrive late and the number is still right.

**Overcapture**

If a second capture occurs before your ISR has cleared `CCyIF`, the hardware sets `CCyOF` as well.

That flag is telling you something important: **you missed an edge**, and the value in `CCRy` is the newer one. If you see `CCyOF` set, your input is faster than your ISR can service, and the measurement is not trustworthy. Check it rather than ignoring it.

The ISR's job is to copy the captured value out of `CCRy` and clear the flag.

# 4 Deep dive 3: The anemometer, and the overflow problem

Recall the cup anemometer from Week 12: magnets on a rotating axle, a reed switch on the mast, one pulse per revolution, with $v_{wind} \approx 2\pi r f_{anem}$.

Listing 1: Book Listing 7.10. Note the AF configuration, which is Week 03 material.

```
#define F_TIM_CLOCK    (48UL*1000UL*1000UL)
#define TIM_PRESCALER (4800)

void Init_TIM_IC(void) {
    RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
    RCC->AHBENR  |= RCC_AHBENR_GPIOBEN;

    /* PB4 -> alternate function 1 = TIM3_CH1 */
    MODIFY_FIELD(GPIOB->MODER,  GPIO_MODER_MODER4, ESF_GPIO_MODER_ALT_FUNC);
    MODIFY_FIELD(GPIOB->AFR[0], GPIO_AFRL_AFSEL4, 1);

    /* time base: full 16-bit range, prescaled */
    TIM3->ARR  = 0xFFFF;
    TIM3->SMCR = 0;
    TIM3->CR1  = 0;                        /* count up */
    TIM3->PSC  = TIM_PRESCALER - 1;

    /* channel 1 as input capture on TI1, rising edge */
    TIM3->CCMR1 |= TIM_CCMR1_CC1S_0;
    TIM3->CCER  |= TIM_CCER_CC1E;

    /* interrupts for BOTH capture and overflow */
    TIM3->DIER |= TIM_DIER_CC1IE | TIM_DIER_UIE;
    TIM3->SR    = 0;

    NVIC_SetPriority(TIM3_IRQn, 128);
    NVIC_ClearPendingIRQ(TIM3_IRQn);
    NVIC_EnableIRQ(TIM3_IRQn);
    TIM3->CR1 |= TIM_CR1_CEN;
}
```

The arithmetic: prescaler 4800 gives a counting frequency of $48\text{ MHz}/4800 = 10$ kHz, so one count is 100 $\mu$s. With `ARR` at `0xFFFF` the overflow frequency is $10\text{ kHz}/65536 = 0.153$ Hz, about once every 6.55 seconds.

## 4.1 The problem: 16 bits is not enough

In light wind the anemometer turns slowly, so **the timer may overflow one or more times between two edges**. Then the naive calculation, current capture minus previous capture, is simply wrong: it computes the difference modulo 65536 and reports a much shorter period, which reports a much higher wind speed.

The fix is the range-extension technique from Week 12, made concrete. Enable the update interrupt too, and count overflows in software. Since `ARR` is `0xFFFF`, each overflow represents exactly `0x10000` $= 65536$ counts, so you can build a **32-bit timestamp** with the overflow count as the upper 16 bits and the captured value as the lower 16:

$$\text{timestamp} = (\text{overflows} \ll 16) \mid \texttt{CCR1}$$

Then take the difference between successive 32-bit timestamps.

**Beware the trap**

**And now the subtle bug, which the book handles explicitly and which is worth studying because the pattern recurs everywhere.**

Both events feed one handler, so a single ISR call may find *both* `UIF` and `CC1IF` set. And you cannot tell from the flags alone which happened **first**.

If the capture came first and the overflow second, then the captured value belongs to the *old* overflow count, and combining it with the new one adds a spurious 65536 counts.

The book's fix is a heuristic on the captured value:

```
if (new_overflow && (TIM3->CCR1 > 0x8000)) {
    extra_overflows = 1;    /* overflow came AFTER the capture */
}
timer_val |= (overflows - extra_overflows) << 16;
```

The reasoning: if the capture value is in the *upper* half of the range, the edge occurred late in the counting cycle, so the overflow that we also see must have happened after it. Subtract one.

This is not a hack, it is the standard way to disambiguate a race between a capture and a wrap, and "**why does combining an overflow count with a capture value need care?" is an excellent exam question**.

Two more details in that ISR worth noting. It detects a stall: if too many overflows pass with no edge, it sets the period to zero to flag invalid data, which is how you distinguish "no wind" from "sensor disconnected". And it defers the floating-point wind-speed calculation to main-line code, passing data through `g_anem_period` with a `g_new_data` flag.

**That deferral is Week 08's partitioning rule, applied**

The ISR does the urgent thing, capturing and differencing integers, and hands off the slow floating-point conversion to `main`. Recall the three options from Week 08: this is the **medium ISR**, and the shared variables must be `volatile`.

On the book's Cortex-M0 the argument is overwhelming, since every float operation is a software library call. On your Cortex-M4F with an FPU it is less dramatic, but recall from Week 10 that floating-point state makes the exception stack frame larger, so it still costs you on every entry.

**In the lab**

Your Lab 06 does frequency measurement with input capture, and its evaluation question asks why the readings fluctuated. This chapter gives you three real answers, and you should give all three:

**1. Switch or sensor bounce.** The book's circuit fits capacitor C1 across the anemometer's reed switch specifically because the contacts bounce on closure, generating extra pulses that the hardware faithfully captures. Every bounce is a false edge and a false period.

**2. Overflow handling.** If the period exceeds the counter's range and you have not extended it in software, readings will be wildly wrong at low frequencies.

**3. Resolution at the wrong end.** Recall Week 12: period measurement has poor relative accuracy for *fast* signals, because the period is only a few counts and $\pm 1$ count is a large fraction. If your fluctuation got worse as frequency rose, that is the cause, and the fix is a smaller prescaler or switching to event counting.

# 5 Deep dive 4: Output compare

Now the other direction. The channel watches for `TIMx_CNT` to match `TIMx_CCRy`, and when it does it can change its output pin, raise an interrupt, or trigger a DMA transfer.

In plain **output compare** mode, the output can be configured to be set to one, cleared to zero, or toggled on each match.

- **Set or clear** produces a single transition. After the first match the output stays put until software changes it, which is how you create a pulse of a fixed duration.
- **Toggle** produces a square wave whose **phase offset** is determined by `CCRy`. Several channels toggling at different `CCRy` values give you several square waves of the same frequency with controlled relative phase.

# 6 Deep dive 5: PWM

The important one. Recall the definition from Week 12: information encoded as duty cycle rather than as a voltage, so it survives noise.

**Where the two numbers come from**

> `TIMx_ARR` **sets the PWM *frequency***, exactly as in Week 12.
> `TIMx_CCRy` **sets the *duty cycle*** for that channel.
>
> So one timer gives you one frequency shared by all its channels, and each channel gets its own independent duty cycle. That is precisely the arrangement you want for driving several motors or dimming several LEDs together.

## 6.1 Edge-aligned

The counter counts up from zero to `ARR`, so the period is $(\texttt{ARR} + 1)/f_{count}$.

Each channel's output is initialised on counter overflow, then changes when `CNT` matches `CCRy`. So:

$$\text{duty cycle} = \frac{\texttt{TIMx\_CCRy}}{\texttt{TIMx\_ARR}}$$

With `ARR`$= 7$ and `CCRy`$= 2$ you get a pulse two counts wide out of eight.

The defining property: **every enabled channel's starting edge is aligned with the counter overflow**, so all channels rise at the same instant.

## 6.2 Centre-aligned

The counter alternates down and up, so the period is $2 \times \texttt{ARR}/f_{count}$.

$$\text{duty cycle} = \frac{2 \times \texttt{CCRy}}{2 \times \texttt{ARR}} = \frac{\texttt{CCRy}}{\texttt{ARR}}$$

Same duty cycle formula, half the frequency for the same `ARR`. Each channel's pulse is **centred** on the counter's transition through zero.

**Why centre-alignment exists**

> Because all edge-aligned channels switch at the same instant, they all draw current at the same instant, which produces a large current spike on the supply rail and correspondingly nasty EMI.
>
> Centre-aligned outputs spread their transitions out in time. The book gives the specific

> motivating case: switching circuits that need a **dead time** between deactivating some components and enabling others, such as a synchronous buck converter or an H-bridge, where turning both halves on simultaneously shorts the supply.
>
> For a robot motor driver, this is why the advanced-control timers exist. Recall from Week 12 that your F303 has **TIM1 and TIM8** with complementary outputs and hardware dead-time insertion, which the book's F091 lacks.

## 6.3 Worked configuration: LED dimming

Target: drive the blue LED on PA7 with PWM.

**Step 1, find the pin's alternate function.** Recall Week 03. The datasheet says PA7 reaches TIM3_CH2 via **AF1**. So the timer and channel are chosen *by the wiring*, not by preference.

**Step 2, choose the frequency.** It must be **at least 50 Hz to avoid visible flicker**. Pick 500 Hz.

**Step 3, do the division.** $48\text{ MHz}/500\text{ Hz} = 96{,}000$. But `ARR` is 16-bit, so the maximum is 65536. **96,000 does not fit**, so a prescaler is required, and it must be at least 2.

Recall the Week 12 rule: prescale as little as possible, because `ARR` is your duty-cycle resolution. So take $\texttt{PSC}+1 = 2$:

$$\texttt{ARR} = \frac{48{,}000{,}000}{500 \times 2} - 1 = 48{,}000 - 1 = 47{,}999$$

48,000 distinct duty-cycle steps. Had you lazily used a prescaler of 96, you would have had 500 steps for the same 500 Hz. Same output frequency, ninety-six times worse resolution.

Listing 2: Book Listing 7.15. Every line earns its place.

```c
#define F_TIM_CLOCK    (48UL*1000UL*1000UL)
#define PWM_FREQUENCY (500)
#define PWM_PRESCALER (2)
#define PWM_MAX_DUTY_VALUE  ((F_TIM_CLOCK/(PWM_FREQUENCY*PWM_PRESCALER))-1)

void Init_Blue_LED_PWM(void) {
    /* PA7 -> AF1 = TIM3_CH2 */
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
    MODIFY_FIELD(GPIOA->MODER,  GPIO_MODER_MODER7, ESF_GPIO_MODER_ALT_FUNC);
    MODIFY_FIELD(GPIOA->AFR[0], GPIO_AFRL_AFRL7, 1);

    /* time base */
    RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
    TIM3->PSC = PWM_PRESCALER - 1;
    TIM3->ARR = PWM_MAX_DUTY_VALUE;
    TIM3->CR1 = 0;                          /* count up, edge aligned */

    /* channel 2 */
    TIM3->CCR2  = 1;                        /* short on-time to start */
    TIM3->CCER |= TIM_CCER_CC2P;            /* ACTIVE LOW polarity    */
    MODIFY_FIELD(TIM3->CCMR1, TIM_CCMR1_OC2M, 6);  /* PWM mode 1     */
    TIM3->CCMR1 |= TIM_CCMR1_OC2PE;         /* enable preload         */
    TIM3->EGR   |= TIM_EGR_UG;              /* force an update        */
    TIM3->CCER  |= TIM_CCER_CC2E;           /* enable channel output  */
    TIM3->BDTR  |= TIM_BDTR_MOE;            /* enable main output     */
    TIM3->CR1   |= TIM_CR1_CEN;             /* start the timer        */
}
```

Four lines deserve comment.

`CC2P` sets **active-low polarity**. Recall Week 03: this LED is wired cathode-to-pin, so a low output lights it. Setting the polarity in hardware means `CCR2 = 0` turns the LED *off* and `CCR2 =`

`ARR` turns it fully on, which is the intuitive direction. Without it, your brightness variable would run backwards.

`OC2PE` enables the **preload register**, and this one matters more than it looks.

**Preload, and why you want it**

> With preload enabled, writing `CCRy` does not take effect immediately. The value is held and transferred into the active register at the **next update event**, meaning the start of the next PWM cycle.
>
> Why that is desirable: if you change the duty cycle mid-cycle without preload, the counter may have *already passed* the new value, so the output never switches and you get one glitched cycle at 100% or 0% duty. On an LED that is a flicker. On a motor it is a current spike.
>
> **Preload makes duty-cycle updates atomic with respect to the PWM period**, which is the same idea as `BSRR` from Week 03 in a different costume. Enable it for any PWM output you intend to change while running.

`EGR |= TIM_EGR_UG` forces an update event immediately, which pushes the preloaded `PSC`, `ARR`, and `CCR` values into their active registers so the first cycle is already correct rather than running once with reset values.

`BDTR |= TIM_BDTR_MOE` enables the main output. On advanced-control timers the outputs are disabled by default as a safety measure, since those timers are intended to drive power stages, and leaving a motor bridge in an undefined state at reset would be dangerous.

**Beware the trap**

> **The commonest PWM failure is a silent one: correct configuration, no signal on the pin.**
>
> Work through the checklist in this order.
> 1. Is the **GPIO clock** on? (Week 03)
> 2. Is `MODER` set to **alternate function**, not output? (Week 03)
> 3. Is `AFSEL` the **right number for that specific pin**? It differs per pin. (Week 03)
> 4. Is the **timer clock** on?
> 5. Is `CCyE` set in `CCER` to enable the channel output?
> 6. Is `MOE` set in `BDTR`, if this is an advanced-control timer?
> 7. Is `CEN` set in `CR1` to actually run the counter?
>
> Seven things, all silent when wrong. This checklist is worth keeping.

The main loop then ramps `CCR2` up and down between 0 and `PWM_MAX_DUTY_VALUE`, with a software delay to make the fade visible. The book notes, correctly and slightly sheepishly, that it would be simple to move the duty adjustment into the timer's ISR and free up most of the CPU. Recall Week 04: that software delay loop is the very thing this chapter exists to eliminate.

**In the lab**

> This is Lab 05 and your self-balancing robot's motor control, so let's do the arithmetic for your hardware.
> Your Balance Car Shield carries a **TB6612FNG** driver, and recall from Week 01 why: the F303's pins cannot drive a gearmotor directly. Your MCU produces logic-level PWM and the TB6612FNG switches the 12 V motor supply.
> **Choosing a PWM frequency for a DC motor** is a real trade-off, unlike the LED case. Too low, and you hear it. Anything under about 20 kHz is in the audible band, which is

why cheap equipment whines.
Too high, and switching losses in the driver rise, and the motor winding inductance has less time to build current per cycle, reducing torque at low duty cycles.
**20 kHz is the standard choice**: just above hearing, comfortably within the TB6612FNG's capability.
At 48 MHz, 20 kHz needs a division of 2400. Following the Week 12 rule, take `PSC`$= 0$ (no prescaling at all) and `ARR` $= 2399$, giving **2400 duty-cycle steps**. That is ample resolution for a PID output.
And note what falls out: your PID controller's output must be scaled to 0–2399, and that scaling factor is `ARR` $+ 1$. If you ever change the PWM frequency, the scaling changes with it. Writing `TIM1->CCR1 = (uint16_t)(pid_out * (ARR+1))` rather than a hard-coded 2400 is the difference between a robot that survives a frequency change and one that flies off the table.

# 7 Deep dive 6: The watchdog

Different kind of timer, same underlying counter.

**Watchdog timer**

A timer that **resets the MCU if software fails to service it** within a required period.

The rationale, stated plainly by the book: it is difficult to make an embedded program perfect. Developers translate ideas into code incorrectly, misunderstanding what a statement means, how a peripheral really operates, or how preemption interacts with shared state. And developers also translate *imperfect ideas*, because humans cannot imagine every possible sequence of inputs, so some are simply absent from the specification.

A watchdog does not fix either kind of bug. It converts an indefinite hang into a bounded outage.

**Book board vs your board**

The book's part has two: the **IWDG** (Independent Watchdog), driven by its own dedicated clock so it **keeps running even if the main clock fails**, and the **WWDG** (System Window Watchdog), driven from the bus clock. The chapter covers the WWDG.

Your F303 has both as well, with the same names and the same distinction. In practice the IWDG is the more robust choice for a deployed system precisely because a main-clock failure is one of the faults you want caught, and the WWDG is the one that catches timing anomalies.

## 7.1 The window, which is the clever part

An ordinary watchdog resets you for being *too late*. A **windowed** watchdog also resets you for being *too early*.

**Why too-early is also a fault**

If your program services the watchdog far sooner than expected, something about its timing has changed. A loop may be spinning without doing its work, or a branch may be skipping the real processing, or an ISR may be firing far more often than intended.

A plain watchdog is perfectly happy with all of those, because it only checks that you are alive. A window checks that you are alive *and running at the expected rate*, which catches

> a whole class of fault the simple version misses.

## 7.2 Registers and the timeout formula

Clock first, as ever: set `WWDGEN` in `RCC_APB1ENR`.

- `WDGA` in `WWDG_CR` activates it. **Once set by software it can only be cleared by a hardware reset**, so a watchdog cannot be switched off by a runaway program.
- `W`, a 7-bit field in `WWDG_CFR`, sets the window. Default `0x7F` means serviceable at any time, so the window is fully open.
- `WDGTB` in `WWDG_CFR` selects the prescaler: bus clock $\div 4096$ (`00`), $\div 2 \times 4096$ (`01`), $\div 4 \times 4096$ (`10`), or $\div 8 \times 4096$ (`11`).
- `T`, the 7-bit counter in `WWDG_CR`. **A reset is generated when it decrements from `0x40` to `0x3F`**, that is, when bit 6 falls from 1 to 0.

$$t_{WWDG} = t_{PCLK} \times 4096 \times 2^{\texttt{WDGTB}} \times (\texttt{T}[5{:}0] + 1)$$

**Worked, from the book**: 48 MHz bus clock, `WDGTB`$= 3$, `T[5:0]`$= 63$ gives a timeout of **43.69 ms**.

Listing 3: Book Listing 7.4. Note that servicing is a single field write.

```
void Init_WWDG(void) {
    RCC->APB1ENR |= RCC_APB1ENR_WWDGEN;
    MODIFY_FIELD(WWDG->CFR, WWDG_CFR_WDGTB, 3);      /* /4096 * 8       */
    MODIFY_FIELD(WWDG->CFR, WWDG_CFR_W,  0x7F);      /* window fully open */
    MODIFY_FIELD(WWDG->CR,  WWDG_CR_T,   0x7F);      /* initial count   */
    WWDG->CR |= WWDG_CR_WDGA;                        /* activate        */
}

void Service_WWDG(void) {
    MODIFY_FIELD(WWDG->CR, WWDG_CR_T, 0x7F);         /* reload counter */
}
```

## 7.3 Finding out why you reset

After a reset, `RCC_CSR` tells you the cause. `WWDGRSTF` is set if the window watchdog did it, and other flags distinguish the independent watchdog, a low-power reset, and the external reset pin. Software clears them by setting `RMVF`.

This is the diagnostics attribute from Week 01 in register form. A deployed device that logs its reset cause tells you whether it crashed or was merely unplugged, and those need very different responses.

## 7.4 Best practices, which are the exam material

**How long should the period be?** The watchdog period sets the **maximum detection delay**, so match it to the consequences of a hang. The book's example is a cordless drill, which needs a short period because a runaway tool can injure the user or damage itself and its battery within a fraction of a second. But a shorter period means servicing more often, which requires genuinely understanding your program's timing behaviour, and that is often more complex than expected.

**Where should it be serviced?**

**Beware the trap**

**Do not service the watchdog in an ISR.**

This is the single most important rule here and the reasoning is elegant: **ISRs are likely to keep running even if the rest of the software has crashed**. A timer interrupt fires whether or not `main` is stuck in an infinite loop. So a watchdog serviced from an ISR will be cheerfully petted by a machine whose actual program died ten minutes ago.

Service it from **main-line, non-ISR code**, preferably low-priority code that is genuinely vulnerable to the crashes you are trying to catch. The whole value of a watchdog is that it is coupled to the health of the thing you care about.

**Do not scatter service calls throughout the code.** Two reasons. It leads to sloppy use that makes the watchdog much less effective, since some path will always reach *a* service call. And multiple service sites make it far harder to work out why a reset happened.

**Startup versus operation.** Embedded programs have two phases. Start-up code configures the system and may contain operations slow enough to need the watchdog serviced several times along the way. Once operational, servicing settles into the main loop.

The book's demo makes this tangible: press B1 and the program stops servicing the watchdog. With $t_{WWDG} = 43.69$ ms the switch must be tapped very quickly indeed to avoid a reset. Their traces show a 49.8 ms press causing a reset and a 19 ms press not.

## 8 Tying it back

We asked what makes a timer more than an alarm clock, and the answer is the channels, in two directions.

**Input capture** latches the counter into `CCRy` the instant an edge arrives, in hardware, so the timestamp is immune to interrupt latency, ISR priority, and masked critical sections, all of which would otherwise appear as jitter in a software reading. Its hazards are **overcapture**, when edges arrive faster than you service them, and the **16-bit range**, which you extend by counting overflows in software and assembling a 32-bit timestamp, taking care to disambiguate whether an overflow arrived before or after a capture.

**Output compare** runs the same comparison the other way, changing a pin when the counter matches, and **PWM** is its most useful form: `ARR` sets the frequency shared by all channels, `CCRy` sets each channel's duty cycle, edge-aligned outputs all start together while centre-aligned ones spread their transitions to allow dead time in power stages. Preload makes duty updates atomic with respect to the PWM period, polarity handles active-low wiring in hardware, and seven separate settings can each silently prevent any output at all.

And the **watchdog** inverts the whole idea: rather than helping software do something on time, it notices when software has stopped. The windowed variant catches running too fast as well as too slow, its activation bit cannot be cleared except by reset, `RCC_CSR` tells you afterwards what happened, and it must be serviced from main-line code because an ISR will keep petting it long after the program it was guarding has died.

Next week the course changes subject entirely. Three weeks of **serial communication**: how you get data off the chip and into another one, one bit at a time, starting with framing and queues and UART.

# Cheat sheet: Week 13

**Channel mechanics**

`TIMx_CCRy` per channel. Channels 1–2 share `CCMR1`, channels 3–4 share `CCMR2`.
`CCyS` selects **input capture** or **output compare** — and **the other** `CCMR` **fields mean different things in each mode**. Read the right row of the bit diagram.
`CCER`: `CCyE` enables the output, `CCyP` sets polarity. `DIER`: `CCyIE` interrupt, `CCyDE` DMA.
`TIMx_SR` holds everything: `UIF` update, `TIF` trigger, `CCyIF` channel event, `CCyOF` overcapture.

**Input capture**

On the chosen edge, hardware copies `CNT` into `CCRy` and sets `CCyIF`.
**Why it beats reading** `CNT` **in an EXTI handler**: a software read includes interrupt latency, instruction completion, higher-priority ISRs, and masked periods — all of which *vary*, so they are jitter, not a calibratable offset. Capture is latched at the edge.
`CCyOF` **means you missed an edge.** Check it; don't ignore it.

**Range extension**: also enable `UIE`, count overflows in software, build a 32-bit timestamp as `(overflows « 16) | CCRy` (valid because `ARR`=`0xFFFF` ⇒ each overflow = 65536 counts).
**The race**: one ISR may see `UIF` and `CC1IF` together and cannot tell the order. If `CCR1 > 0x8000` the capture came first, so subtract one overflow.

**Three causes of fluctuating frequency readings**: switch/sensor **bounce** (fit a capacitor), **unhandled overflow** at low frequency, $\pm 1$ **count resolution** at high frequency.

**PWM**

`ARR` **sets the frequency** (shared by all channels). `CCRy` **sets the duty cycle** (per channel).

**Edge-aligned**: counts 0 up to `ARR`. Period = $(\texttt{ARR}+1)/f_{count}$. Duty = `CCRy`/`ARR`. **All channels rise together.**
**Centre-aligned**: counts up and down. Period = $2\texttt{ARR}/f_{count}$. Duty = `CCRy`/`ARR`. Pulses centred on the zero crossing, which **allows dead time** for H-bridges and buck converters.

**Worked**: 500 Hz from 48 MHz needs ÷96,000, which exceeds 16-bit `ARR`. So `PSC`$+1 = 2$, `ARR`$= 47{,}999 \Rightarrow$ 48,000 duty steps. Using `PSC`$= 95$ instead gives the same 500 Hz with only 500 steps.
LED PWM must be $\geq$ **50 Hz** to avoid visible flicker.

**PWM configuration details**

`CCyP` — **active-low polarity** in hardware, so `CCRy=0` is off even with a cathode-to-pin LED.
`OCyPE` — **preload**: `CCRy` writes take effect at the *next update event*, not immediately. Without it, changing duty mid-cycle can glitch a whole cycle to 0% or 100%. **Makes duty updates atomic w.r.t. the PWM period.**
`EGR |= UG` — force an update so preloaded `PSC`/`ARR`/`CCR` take effect before the first cycle.
`BDTR |= MOE` — main output enable, off by default on advanced timers as a power-stage safety measure.

**No PWM on the pin? Check all seven:** GPIO clock · `MODER`=alt func · correct `AFSEL` for that pin · timer clock · `CCyE` · `MOE` · `CEN`.

**Motor PWM**: **20 kHz** — above hearing, below excessive switching loss. At 48 MHz: `PSC`$= 0$, `ARR`$= 2399$.

**Watchdog**

Resets the MCU if software fails to service it. Doesn't fix bugs; **converts an indefinite hang into a bounded outage**.
**IWDG**: own dedicated clock, **survives main clock failure**. **WWDG**: bus clock, **windowed**. **Windowed** = resets you for being too *early* as well as too late, catching "running at the wrong rate" faults a plain watchdog misses.

`WDGA` activates it and **can only be cleared by a hardware reset**. `W` sets the window (`0x7F` = fully open). `WDGTB` prescales by $4096 \times 2^{\texttt{WDGTB}}$. `T` is the 7-bit counter; **reset fires when it goes** `0x40` → `0x3F`.

$$t_{WWDG} = t_{PCLK} \times 4096 \times 2^{\texttt{WDGTB}} \times (\texttt{T}[5{:}0] + 1)$$

48 MHz, `WDGTB`= 3, `T[5:0]`= 63 ⇒ **43.69 ms**.
`RCC_CSR` gives the reset cause (`WWDGRSTF` etc.); clear with `RMVF`.

**Watchdog best practices — these are short-answer marks**

**Period** sets the **maximum detection delay**. Short period for things that can cause harm fast (the book's cordless drill). Shorter period ⇒ service more often ⇒ needs real understanding of your timing.
**Never service it in an ISR — ISRs keep running even when the rest of the software has crashed**, so the watchdog would be petted by a dead program. Service from **main-line, low-priority code**.
**Don't scatter service calls**: it makes the watchdog ineffective and makes reset causes untraceable.
Start-up code may need to service it several times; operational code settles into the main loop.

**Likely exam prompts from this section**

· Given a target PWM frequency and duty cycle, compute `PSC`, `ARR`, and `CCRy`. *Near certain.*
· Distinguish capture mode from compare mode, with a use case for each. *This is your Lab 06 question.*
· Why is input capture more accurate than reading the counter in an ISR?
· Why does combining an overflow count with a captured value need care?
· Distinguish edge-aligned from centre-aligned PWM and say when you need the latter.
· Compute a WWDG timeout, or find the settings for the shortest/longest one.
· Why should a watchdog never be serviced from an ISR?
· List reasons an input-capture frequency measurement might fluctuate.